

MANUAL MAESTRO DE PROGRAMACIÓN EN PYTHON

De la Sintaxis Fundamental a la Computación Científica con NumPy

Guía Teórico-Práctica y Manual de Referencia

Incluye explicaciones detalladas, formulación matemática analítica, ejemplos de código ejecutables, patrones algorítmicos, buenas prácticas (PEP 8) y proyecto integrador.

Ejes Temáticos Cubiertos:

- Formateo con **f-strings**
- Definición de Funciones (**def**)
- Cálculo de Áreas Geométricas
- Volúmenes de Cuerpos Sólidos
- Listas y Métodos Dinámicos
- Manipulación Avanzada de Cadenas
- Estructuras Condicionales (**if**)
- Bucles y Patrones con **for** y **while**
- Biblioteca Científica NumPy
- Proyecto y Resumen Integrador

Asignatura: Programación e Ingeniería Computacional 2026-2

Fecha de Publicación: Septiembre de 2026

Compilador: Tectonic LaTeX Engine / Python 3.10+

Índice

1. Cadenas Literales Formateadas: f-strings	2
1.1. Sintaxis Básica e Interpolación de Expresiones	2
1.2. Especificadores de Formato Numérico y Alineación	2
2. Definición de Funciones (def)	4
2.1. Estructura, Parámetros y Valores de Retorno	4
2.2. Parámetros por Defecto, *args y **kwargs	4
3. Cálculos Básicos: Área de Rectángulos y Triángulos	6
4. Cálculo de Volúmenes de Sólidos Geométricos	8
5. Creación y Manipulación de Listas (list)	10
5.1. Indexación, Slicing y Operaciones Esenciales	10
5.2. Comprensión de Listas (<i>List Comprehensions</i>)	10
6. Manipulación de Cadenas de Caracteres (str)	12
6.1. Métodos Principales de Transformación y Limpieza	12
7. Estructuras de Control Condicional (if, elif, else)	14
7.1. Operadores Relacionales, Lógicos y de Pertenencia	14
7.2. Operador Ternario	15
8. Bucles y Construcción de Patrones usando for i	16
8.1. Control de Iteración: range(), enumerate() y zip()	16
8.2. Generación de Patrones Geométricos y Numéricos	16
9. Computación Científica con el Paquete NumPy	18
9.1. Creación e Inicialización de Arreglos (ndarray)	18
9.2. Operaciones Vectorizadas y Broadcasting	18
9.3. Indexación Avanzada, Slicing y Máscaras Booleanas	19
9.4. Estadística Descriptiva y Álgebra Lineal	19
10. Proyecto Integrador: Sistema de Análisis Geométrico y Estadístico	21
10.1. Cuadro Resumen de Sintaxis y Referencia Rápida	24

1 Cadenas Literales Formateadas: f-strings

¿Qué son las f-strings?

Introducidas en Python 3.6 (PEP 498), las **f-strings** (*Formatted String Literals*) proporcionan una sintaxis concisa, altamente legible y eficiente para interpolar variables y evaluar expresiones dentro de cadenas de texto. Se definen prefijando la cadena con la letra **f** o **F** y delimitando las expresiones entre llaves `{...}`. A diferencia de los métodos heredados (`% operator` o `str.format()`), las f-strings se evalúan en tiempo de ejecución a código de byte directo, haciéndolas significativamente más rápidas.

1.1 Sintaxis Básica e Interpolación de Expresiones

Cualquier expresión válida de Python puede residir dentro de las llaves, incluyendo operaciones aritméticas, llamadas a funciones y métodos de objetos.

Código Python: Interpolación Básica y Expresiones Dinámicas

```
1 nombre = "Carlos"
2 edad = 20
3 nota1 = 4.5
4 nota2 = 3.8
5
6 # Interpolación directa de variables
7 print(f"Hola, {nombre}. Tienes {edad} años.")
8
9 # Evaluación de expresiones matemáticas dentro de la f-string
10 print(f"El promedio final es: {(nota1 + nota2) / 2}")
11
12 # Llamadas a métodos y transformaciones directas
13 print(f"Nombre en mayúsculas: {nombre.upper()} | Caracteres:
    {len(nombre)}")
```

Salida en Consola

```
Hola, Carlos. Tienes 20 años.
El promedio final es: 4.15
Nombre en mayúsculas: CARLOS | Caracteres: 6
```

1.2 Especificadores de Formato Numérico y Alineación

El operador dos puntos `:` dentro de las llaves permite aplicar especificadores de formato según la siguiente convención:

Código Python: Ejemplo de Tabulación y Formato Financiero/Científico

```
1 producto = "Sensor LiDAR"
2 precio = 1850.758
3 descuento = 0.15
4 stock = 7
```

Especificador	Descripción	Ejemplo con $x = 1234.5678$
{x:.2f}	Punto flotante con 2 decimales	"1234.57"
{x:,.2f}	Separador de miles y 2 decimales	"1,234.57"
{x:.2e}	Notación científica exponencial	"1.23e+03"
{x:>12.2f}	Alineación derecha en ancho de 12 caracteres	"1234.57"
{x:<12.2f}	Alineación izquierda en ancho de 12 caracteres	"1234.57 "
{x:^12.2f}	Centrado en ancho de 12 caracteres	"1234.57 "
{p:.1%}	Formato de porcentaje ($p = 0.854$)	"85.4%"
{n:06d}	Entero relleno con ceros a 6 dígitos ($n = 42$)	"000042"
{n:b} / {n:x}	Representación Binaria y Hexadecimal ($n = 255$)	"11111111/ "ff"

Tabla 1: Especificadores de formato comunes en f-strings.

```

5
6 # Imprimir reporte formateado con alineacion
7 print(f"Producto: {producto:<20} | Stock: {stock:03d}")
8 print(f"Precio Unitario: ${precio:,.2f}")
9 print(f"Descuento:          {descuento:.1%}")
10 print(f"Total a Pagar:      ${precio * (1 - descuento):,.2f}")
11
12 # Depuracion rapida (Python 3.8+) mediante el sufijo '='
13 x = 42
14 y = 13.5
15 print(f"{x=}, {y=}, {x*y=}")

```

Salida en Consola

```

Producto: Sensor LiDAR          | Stock: 007
Precio Unitario: $1,850.76
Descuento:          15.0%
Total a Pagar:      $1,573.14
x=42, y=13.5, x*y=567.0

```

Escape de Llaves

Para imprimir una llave literal { o } dentro de una f-string, se debe duplicar: `f"{{texto}}"` producirá en pantalla `"{texto}"`.

2 Definición de Funciones (def)

Modularidad y Principio DRY

Una función es un bloque de código reutilizable y parametrizado diseñado para ejecutar una tarea específica. Promueve el principio **DRY** (*Don't Repeat Yourself*), facilitando la legibilidad, pruebas unitarias y mantenimiento del software.

2.1 Estructura, Parámetros y Valores de Retorno

En Python, una función se declara con la palabra reservada `def`, seguida del nombre de la función, sus parámetros entre paréntesis y dos puntos (`:`). El cuerpo de la función debe estar indentado (estándar PEP 8: 4 espacios).

Código Python: Anatomía Completa de una Función

```
1 def calcular_estadisticas(valores: list[float]) -> tuple[float,
2     float, float]:
3     """
4     Calcula la suma, promedio y rango de una lista de numeros.
5
6     Parametros:
7         valores (list[float]): Lista con datos numericos.
8
9     Retorna:
10        tuple[float, float, float]: (suma, promedio, rango)
11    """
12    if not valores:
13        raise ValueError("La lista de valores no puede estar vacia.")
14
15    total = sum(valores)
16    promedio = total / len(valores)
17    rango = max(valores) - min(valores)
18
19    return total, promedio, rango # Retorno empaquetado en tupla
20
21 # Invocacion y desempacado de valores
22 datos = [12.5, 18.0, 9.5, 24.0, 15.0]
23 s, p, r = calcular_estadisticas(datos)
24 print(f"Suma: {s:.2f} | Promedio: {p:.2f} | Rango: {r:.2f}")
```

2.2 Parámetros por Defecto, *args y **kwargs

Python permite definir argumentos posicionales, argumentos con valores predeterminados y parámetros de longitud variable para máxima flexibilidad.

Código Python: Flexibilidad en Parámetros y Argumentos Variables

```
1 # Parametro con valor por defecto
2 def saludar(nombre: str, titulo: str = "Ing.", saludo: str =
3     "Bienvenido") -> str:
```

```
3     return f"{saludo}, {titulo} {nombre}."
4
5     # *args (tupla de argumentos posicionales variables)
6     # **kwargs (diccionario de argumentos nombrados variables)
7     def registrar_experimento(id_exp: int, *mediciones: float,
8                               **metadatos) -> None:
9         print(f"\n--- Registro del Experimento #{id_exp} ---")
10        if mediciones:
11            print(f"Mediciones ({len(mediciones)}): {mediciones}")
12            print(f"Promedio de medicion:
13                  {sum(mediciones)/len(mediciones):.4f}")
14        for clave, valor in metadatos.items():
15            print(f" {clave.capitalize()}: {valor}")
16
17    print(saludar("Rodriguez"))
18    print(saludar("Lopez", titulo="Dra.", saludo="Buenas tardes"))
19
20    registrar_experimento(101, 4.12, 4.15, 4.09, 4.18,
21                          laboratorio="Fisica Cuantica",
22                          responsable="Dra. Lopez",
23                          temperatura_C=22.5)
```

Salida en Consola

```
Bienvenido, Ing. Rodriguez.
Buenas tardes, Dra. Lopez.

--- Registro del Experimento #101 ---
Mediciones (4): (4.12, 4.15, 4.09, 4.18)
Promedio de medicion: 4.1350
Laboratorio: Fisica Cuantica
Responsable: Dra. Lopez
Temperatura_c: 22.5
```

3 Cálculos Básicos: Área de Rectángulos y Triángulos

En ingeniería y ciencias computacionales, el cálculo de áreas planas es uno de los problemas fundamentales. A continuación se formulan analíticamente e implementan de forma modular.

Área del Rectángulo

Para un rectángulo de base b y altura h :

$$A_{\text{rect}} = b \cdot h \quad (1)$$

Área del Triángulo (Base y Altura)

Para un triángulo de base b y altura h :

$$A_{\text{triang}} = \frac{b \cdot h}{2} \quad (2)$$

Fórmula de Herón (Triángulo conociendo sus tres lados)

Para cualquier triángulo de lados a, b, c , el semiperímetro s y el área A se determinan mediante:

$$s = \frac{a + b + c}{2}, \quad A_{\text{Heron}} = \sqrt{s(s-a)(s-b)(s-c)} \quad (3)$$

Condición de existencia geométrica (Desigualdad Triangular): $a + b > c$, $a + c > b$, y $b + c > a$.

Código Python: Módulo de Cálculo de Áreas con Validaciones Robustas

```
1 import math
2
3 def area_rectangulo(base: float, altura: float) -> float:
4     """Calcula el area de un rectangulo validando datos positivos."""
5     if base <= 0 or altura <= 0:
6         raise ValueError("La base y la altura deben ser valores
7             positivos.")
8     return base * altura
9
10 def area_triangulo_base_altura(base: float, altura: float) -> float:
11     """Calcula el area de un triangulo a partir de su base y
12         altura."""
13     if base <= 0 or altura <= 0:
14         raise ValueError("La base y la altura deben ser valores
15             positivos.")
16     return (base * altura) / 2.0
17
18 def area_triangulo_heron(a: float, b: float, c: float) -> float:
19     """Calcula el area de un triangulo usando la Formula de Heron."""
20     if a <= 0 or b <= 0 or c <= 0:
21         raise ValueError("Los lados deben ser estrictamente
22             positivos.")
23     # Validacion de la desigualdad triangular
24     if (a + b <= c) or (a + c <= b) or (b + c <= a):
25         raise ValueError("Los lados proporcionados no forman un
26             triangulo geometrico valido.")
27
28     s = (a + b + c) / 2.0
29     area = math.sqrt(s * (s - a) * (s - b) * (s - c))
30     return area
31
32 # Demostracion y ejecucion
33 b_rect, h_rect = 8.5, 4.2
```

```
29 b_tri, h_tri = 6.0, 7.5
30 l1, l2, l3 = 5.0, 6.0, 7.0
31
32 print(f"Area del Rectangulo ({b_rect}m x {h_rect}m):
      {area_rectangulo(b_rect, h_rect):.4f} m^2")
33 print(f"Area del Triangulo (base={b_tri}m, h={h_tri}m):
      {area_triangulo_base_altura(b_tri, h_tri):.4f} m^2")
34 print(f"Area Triangulo Heron (lados={l1}, {l2}, {l3}):
      {area_triangulo_heron(l1, l2, l3):.4f} m^2")
```

Salida en Consola

```
Area del Rectangulo (8.5m x 4.2m):    35.7000 m^2
Area del Triangulo (base=6.0m, h=7.5m):  22.5000 m^2
Area Triangulo Heron (lados=5.0, 6.0, 7.0): 14.6969 m^2
```


4 Cálculo de Volúmenes de Sólidos Geométricos

El cálculo volumétrico es esencial en modelos de física, mecánica de fluidos y diseño computacional.

Cuerpo Sólido	Fórmula Matemática	Variables / Parámetros
Cilindro Circular	$V = \pi r^2 h$	Radio de la base r , Altura h
Esfera	$V = \frac{4}{3} \pi r^3$	Radio r
Prisma Rectangular	$V = l \cdot w \cdot h$	Longitud l , Ancho w , Altura h
Cono Circular Recto	$V = \frac{1}{3} \pi r^2 h$	Radio de la base r , Altura h
Pirámide Cuadrada	$V = \frac{1}{3} L^2 h$	Lado de la base L , Altura h

Tabla 2: Fórmulas analíticas de volumen para sólidos comunes.

Código Python: Módulo de Volúmenes con math y Arquitectura Modular

```
1 import math
2
3 class SolidosGeometricos:
4     @staticmethod
5     def volumen_cilindro(radio: float, altura: float) -> float:
6         """Calcula el volumen de un cilindro:  $V = \pi * r^2 * h$ """
7         assert radio > 0 and altura > 0, "Dimensiones deben ser mayores a cero"
8         return math.pi * (radio ** 2) * altura
9
10    @staticmethod
11    def volumen_esfera(radio: float) -> float:
12        """Calcula el volumen de una esfera:  $V = (4/3) * \pi * r^3$ """
13        assert radio > 0, "El radio debe ser mayor a cero"
14        return (4.0 / 3.0) * math.pi * (radio ** 3)
15
16    @staticmethod
17    def volumen_prisma_rectangular(largo: float, ancho: float, alto: float) -> float:
18        """Calcula el volumen de un paralelepipedo:  $V = l * w * h$ """
19        assert largo > 0 and ancho > 0 and alto > 0, "Dimensiones invalidas"
20        return largo * ancho * alto
21
22    @staticmethod
23    def volumen_cono(radio: float, altura: float) -> float:
24        """Calcula el volumen de un cono circular:  $V = (1/3) * \pi * r^2 * h$ """
25        assert radio > 0 and altura > 0, "Dimensiones invalidas"
26        return (1.0 / 3.0) * math.pi * (radio ** 2) * altura
27
28    # Evaluacion comparativa
29    r, h = 3.5, 10.0
30    vol_cil = SolidosGeometricos.volumen_cilindro(r, h)
31    vol_esf = SolidosGeometricos.volumen_esfera(r)
32    vol_con = SolidosGeometricos.volumen_cono(r, h)
33    vol_pri = SolidosGeometricos.volumen_prisma_rectangular(4.0, 5.0, 6.0)
34
35    print(f"{'Sólido':<20} | {'Parámetros':<22} | {'Volumen Calculado':>18}")
36    print("-" * 66)
```

```
37 print(f"{'Cilindro':<20} | {f'r={r}, h={h}':<22} | {vol_cil:>16.4f} u^3")
38 print(f"{'Esfera':<20} | {f'r={r}':<22} | {vol_esf:>16.4f} u^3")
39 print(f"{'Cono':<20} | {f'r={r}, h={h}':<22} | {vol_con:>16.4f} u^3")
40 print(f"{'Prisma Rectangular':<20} | {f'4.0 x 5.0 x 6.0':<22} | {vol_pri:>16.4f} u^3")
```

Salida en Consola

Solido	Parametros	Volumen Calculado
Cilindro	r=3.5, h=10.0	384.8451 u^3
Esfera	r=3.5	179.5944 u^3
Cono	r=3.5, h=10.0	128.2817 u^3
Prisma Rectangular	4.0 x 5.0 x 6.0	120.0000 u^3

5 Creación y Manipulación de Listas (list)

Naturaleza de las Listas en Python

Las listas en Python son secuencias **mutables**, **ordenadas** y **heterogéneas** que almacenan referencias a objetos en memoria contigua redimensionable.

5.1 Indexación, Slicing y Operaciones Esenciales

El acceso a elementos se realiza mediante corchetes [], admitiendo indexación positiva (0 a $n - 1$) y negativa (-1 para el último elemento). El rebanado (*slicing*) sigue la regla [inicio:fin:passo].

Código Python: Operaciones y Métodos Nativos de Listas

```

1  # Creacion de listas
2  numeros = [10, 25, 4, 89, 33, 56, 7]
3
4  # Indexacion y slicing
5  print(f"Primer elemento: {numeros[0]} | Ultimo elemento:
      {numeros[-1]}")
6  print(f"Sublista (pos 1 a 4): {numeros[1:5]}")
7  print(f"Sublista invertida: {numeros[::-1]}")
8  print(f"Elementos en indices pares: {numeros[::2]}")
9
10 # Mutabilidad y modificacion dinamica
11 numeros.append(100)          # Agrega al final
12 numeros.insert(2, 999)       # Inserta 999 en el indice 2
13 numeros.extend([50, 60])     # Extiende con otra lista
14 eliminado = numeros.pop()    # Extrae y retorna el ultimo elemento
15 numeros.remove(999)          # Elimina la primera aparicion de 999
16
17 print(f"Lista tras modificaciones: {numeros}")
18 print(f"Elemento retirado con pop(): {eliminado}")
19
20 # Ordenamiento
21 numeros.sort()               # Orden in-situ ascendente
22 print(f"Lista ordenada: {numeros}")

```

Salida en Consola

```

Primer elemento: 10 | Ultimo elemento: 7
Sublista (pos 1 a 4): [25, 4, 89, 33]
Sublista invertida: [7, 56, 33, 89, 4, 25, 10]
Elementos en indices pares: [10, 4, 33, 7]
Lista tras modificaciones: [10, 25, 4, 89, 33, 56, 7, 100, 50]
Elemento retirado con pop(): 60
Lista ordenada: [4, 7, 10, 25, 33, 50, 56, 89, 100]

```

5.2 Comprensión de Listas (List Comprehensions)

Las comprensiones de listas permiten generar y transformar secuencias de manera concisa y altamente optimizada.

Código Python: Comprensión de Listas y Filtrado Condicional

```
1 # Sintaxis: [expresion for elemento in iterable if condicion]
2
3 # 1. Cuadrados de numeros pares del 1 al 12
4 cuadrados_pares = [x ** 2 for x in range(1, 13) if x % 2 == 0]
5 print(f"Cuadrados de pares: {cuadrados_pares}")
6
7 # 2. Conversion de temperaturas Celsius a Fahrenheit
8 temps_celsius = [0.0, 15.5, 22.0, 30.0, 38.5]
9 temps_fahrenheit = [(c * 9/5) + 32 for c in temps_celsius]
10 print(f"Temperaturas en F: {[round(f, 2) for f in temps_fahrenheit]}")
11
12 # 3. Matriz 2D generada por comprension anidada (matriz identidad 3x3)
13 matriz_identidad = [[1 if i == j else 0 for j in range(3)] for i in
14                       range(3)]
15 print("Matriz Identidad 3x3:")
16 for fila in matriz_identidad:
17     print(f" {fila}")
```

Salida en Consola

```
Cuadrados de pares: [4, 16, 36, 64, 100, 144]
Temperaturas en F: [32.0, 59.9, 71.6, 86.0, 101.3]
Matriz Identidad 3x3:
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]
```

6 Manipulación de Cadenas de Caracteres (str)

Inmutabilidad de los Strings

En Python, los strings son secuencias **inmutables** de caracteres Unicode. Cualquier método que opere sobre una cadena retorna un **nuevo** objeto string en memoria sin alterar el original.

6.1 Métodos Principales de Transformación y Limpieza

Categoría	Método	Función y Efecto
Casing	<code>.upper()</code> , <code>.lower()</code> <code>.title()</code> , <code>.capitalize()</code>	Convierte a mayúsculas o minúsculas sostenidas Capitaliza cada palabra o solo el primer carácter
Limpieza	<code>.strip()</code> , <code>.rstrip()</code>	Elimina espacios o caracteres en blanco en extremos
Búsqueda	<code>.find(sub)</code> , <code>.count(sub)</code> <code>.startswith()</code> , <code>.endswith()</code>	Retorna el índice de coincidencia o número de apariciones Verifica prefijo o sufijo booleano
División y Unión	<code>.split(sep)</code> , <code>.join(iter)</code> <code>.replace(ant, nue)</code>	Divide cadena en lista o une elementos con un delimitador Reemplaza subcadenas

Tabla 3: Principales métodos integrados del tipo `str`.

Código Python: Ejemplos Prácticos de Procesamiento y Parseo de Cadenas

```

1 # Registro crudo proveniente de telemetria
2 registro_crudo = "    ID_SENSOR:SN-4891;VAR:PRESION;VAL:101.325;EST:OK\n"
3
4 # 1. Limpieza y separacion
5 limpio = registro_crudo.strip()
6 campos = limpio.split(";")
7 print(f"Campos extraidos: {campos}")
8
9 # 2. Parseo y extraccion de valores en un diccionario
10 datos_sensor = {}
11 for campo in campos:
12     clave, valor = campo.split(":")
13     datos_sensor[clave.lower()] = valor
14
15 print(f"Diccionario parseado: {datos_sensor}")
16 print(f"Estado en minuscula:  {datos_sensor['est'].lower()}")
17 print(f"Valor numerico float: {float(datos_sensor['val']):.2f} kPa")
18
19 # 3. Construcccion de cadena estructurada con .join()
20 elementos = ["Python", "C++", "Rust", "Julia", "Fortran"]
21 lenguajes_csv = " -> ".join(elementos)
22 print(f"Cadena unificada: {lenguajes_csv}")
23
24 # 4. Inversion de cadenas mediante slicing
25 palabra = "Reconocimiento"
```

```
26 print(f"Original: {palabra} | Invertida: {palabra[::-1]}")
```

Salida en Consola

```
Campos extraídos: ['ID_SENSOR:SN-4891', 'VAR:PRESION', 'VAL:101.325', 'EST:OK']
Diccionario parseado: {'id_sensor': 'SN-4891', 'var': 'PRESION', 'val':
    '101.325', 'est': 'OK'}
Estado en minúscula: ok
Valor numérico float: 101.32 kPa
Cadena unificada: Python -> C++ -> Rust -> Julia -> Fortran
Original: Reconocimiento | Invertida: otneimiconoceR
```

7 Estructuras de Control Condicional (if, elif, else)

Bifurcación de Flujo Lógico

Las sentencias condicionales permiten evaluar expresiones booleanas (True o False) para bifurcar el flujo de ejecución del programa en función del estado de las variables y condiciones del entorno.

7.1 Operadores Relacionales, Lógicos y de Pertenencia

- **Relacionales:** == (igualdad), != (diferente), <, >, <=, >=.
- **Lógicos:** and (conjunción estricta), or (disyunción), not (negación).
- **Pertenencia e Identidad:** in, not in, is, is not.

Código Python: Estructura Condicional Completa y Clasificador Físico

```

1 def clasificar_estado_materia(temp_celsius: float) -> str:
2     """Clasifica el estado del agua a 1 atm segun la temperatura."""
3     if temp_celsius < 0.0:
4         estado = "Sólido (Hielo)"
5     elif 0.0 ≤ temp_celsius < 100.0:
6         estado = "Líquido"
7     elif 100.0 ≤ temp_celsius < 1000.0:
8         estado = "Gaseoso (Vapor de agua)"
9     else:
10        estado = "Plasma termico"
11    return estado
12
13 def evaluar_acceso(usuario: str, rol: str, activo: bool, intentos:
14    int) -> str:
15    # Evaluacion con operadores logicos combinados y cortocircuito
16    if not activo or intentos ≥ 3:
17        return "ACCESO DENEGADO: Cuenta bloqueada o inactiva."
18
19    if rol in ["Administrador", "Superusuario"] and (usuario ==
20        "root" or usuario.startswith("admin_")):
21        return f"ACCESO TOTAL CONCEDIDO a {usuario} ({rol})."
22    elif rol == "Investigador":
23        return f"ACCESO DE CONSULTA CONCEDIDO a {usuario}."
24    else:
25        return "ACCESO RESTRINGIDO: Rol insuficiente."
26
27 # Pruebas del sistema condicional
28 temperaturas = [-5.2, 22.0, 100.0, 1500.0]
29 for t in temperaturas:
30     print(f"Temp: {t:>6.1f} C -> Estado:
31         {clasificar_estado_materia(t)}")
32
33 print()
34 print(evaluar_acceso("admin_juan", "Administrador", activo=True,
35     intentos=0))
36 print(evaluar_acceso("carlos_inv", "Investigador", activo=True,
37     intentos=1))

```

```
33 print(evaluar_acceso("hacker_01", "Invitado", activo=True,
    intentos=4))
```

7.2 Operador Ternario

Python proporciona una expresión condicional ternaria: `valor_verdadero if condicion else valor_falso`.

Código Python: Expresión Ternaria Concisa

```
1 voltaje = 12.4
2 umbral = 11.0
3
4 # Operador ternario conciso
5 estado_bateria = "Optima" if voltaje ≥ umbral else "Baja (Cargar)"
6 print(f"Voltaje: {voltaje}V | Estado: {estado_bateria}")
```

Salida en Consola

```
Temp:   -5.2 C -> Estado: Solido (Hielo)
Temp:   22.0 C -> Estado: Liquido
Temp:  100.0 C -> Estado: Gaseoso (Vapor de agua)
Temp: 1500.0 C -> Estado: Plasma termico

ACCESO TOTAL CONCEDIDO a admin_juan (Administrador).
ACCESO DE CONSULTA CONCEDIDO a carlos_inv.
ACCESO DENEGADO: Cuenta bloqueada o inactiva.
Voltaje: 12.4V | Estado: Optima
```


8 Bucles y Construcción de Patrones usando for i

La Función range() y Bucles for

El bucle for itera sobre elementos de cualquier secuencia o generador. La función range(inicio, fin, paso) produce una progresión aritmética que inicia en inicio, se incrementa en paso y finaliza estrictamente antes de alcanzar fin.

8.1 Control de Iteración: range(), enumerate() y zip()

Código Python: Iteración con Enumeración e Iteración Paralela

```
1 nombres = ["Ana", "Bernardo", "Catalina", "Diego"]
2 puntajes = [92.5, 88.0, 95.0, 79.5]
3
4 # Iteracion con índice y valor simultaneos via enumerate()
5 print("Listado de Estudiantes:")
6 for i, nombre in enumerate(nombres, start=1):
7     print(f" Posicion {i}: {nombre}")
8
9 # Iteracion paralela de multiples listas via zip()
10 print("\nBoletin de Notas:")
11 for nombre, nota in zip(nombres, puntajes):
12     estado = "Aprobado" if nota ≥ 80.0 else "Requiere Nivelacion"
13     print(f" {nombre:<10}: {nota:>5.1f} pts -> {estado}")
```

8.2 Generación de Patrones Geométricos y Numéricos

El anidamiento de bucles (for i exterior para las filas y for j interior para las columnas) permite generar matrices de caracteres, pirámides y patrones gráficos en consola.

Código Python: Catálogo de Patrones con Bucles Anidados

```
1 n = 5
2
3 print("1. Triangulo Rectangulo Creciente:")
4 for i in range(1, n + 1):
5     print("* " * i)
6
7 print("\n2. Triangulo de Numeros Consecutivos:")
8 for i in range(1, n + 1):
9     for j in range(1, i + 1):
10        print(j, end=" ")
11    print()
12
13 print("\n3. Piramide Centrada (Triangulo Equilatero):")
14 for i in range(1, n + 1):
15     espacios = " " * (n - i)
16     estrellas = "* " * i
17     print(f"{espacios}{estrellas}")
18
```

```

19 print("\n4. Rombo / Diamante Completo:")
20 # Parte superior creciente
21 for i in range(1, n + 1):
22     print(" " * (n - i) + "*" * i)
23 # Parte inferior decreciente
24 for i in range(n - 1, 0, -1):
25     print(" " * (n - i) + "*" * i)
26
27 print("\n5. Matriz de Coordenadas (i, j):")
28 for i in range(1, 4):
29     for j in range(1, 4):
30         print(f"({i},{j})", end=" ")
31     print()

```

Salida en Consola

```

1. Triangulo Rectangulo Creciente:
*
* *
* * *
* * * *
* * * * *

2. Triangulo de Numeros Consecutivos:
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

3. Piramide Centrada:
  *
  * *
 * * *
* * * *
* * * * *

4. Rombo / Diamante Completo:
  *
  * *
 * * *
* * * *
* * * * *
 * * * *
  * * *
   * *
    *

5. Matriz de Coordenadas:
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)

```

9 Computación Científica con el Paquete NumPy

¿Por qué NumPy es el Núcleo del Ecosistema Científico?

NumPy (*Numerical Python*) proporciona el objeto `ndarray` (N -dimensional array), una estructura de datos en memoria contigua de C optimizada para operaciones vectorizadas y álgebra lineal. Es órdenes de magnitud más rápido que las listas estándar de Python al evitar la sobrecarga del tipado dinámico e interpretar bucles a nivel de hardware mediante instrucciones vectoriales SIMD.

9.1 Creación e Inicialización de Arreglos (`ndarray`)

Código Python: Constructores Principales de Arrays

```
1 import numpy as np
2
3 # 1. Desde listas nativas
4 arr_1d = np.array([10, 20, 30, 40, 50], dtype=np.float64)
5 arr_2d = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)
6
7 # 2. Arreglos especiales predefinidos
8 ceros = np.zeros((2, 3))           # Matriz 2x3 de ceros
9 unos = np.ones((3, 2))            # Matriz 3x2 de unos
10 identidad = np.eye(3)             # Matriz Identidad 3x3
11
12 # 3. Secuencias numericas continuas
13 rango_np = np.arange(0, 10, 2)     # [0, 2, 4, 6, 8]
14 lineal = np.linspace(0.0, 1.0, 5)  # 5 puntos equiespaciados entre
    0 y 1
15
16 # 4. Generacion de numeros aleatorios
17 np.random.seed(42)                 # Semilla para reproducibilidad
18 aleatorios = np.random.uniform(10.0, 20.0, size=(2, 2))
19
20 print(f"Shape arr_2d: {arr_2d.shape} | Dimensiones: {arr_2d.ndim} |
    Tipo: {arr_2d.dtype}")
21 print(f"Linspace (0 a 1, 5 pts): {lineal}")
22 print(f"Matriz Aleatoria Uniforme:\n{aleatorios}")
```

9.2 Operaciones Vectorizadas y Broadcasting

En NumPy, los operadores aritméticos se aplican **elemento a elemento** sin necesidad de bucles explícitos. El *Broadcasting* permite operar arreglos con diferentes formas compatibles.

Código Python: Aritmética Vectorizada y Funciones Universales (ufuncs)

```
1 x = np.array([0.0, np.pi/6, np.pi/4, np.pi/2])
2
3 # Operaciones trigonometricas y trascendentes vectorizadas
4 seno_x = np.sin(x)
5 coseno_x = np.cos(x)
```

```
6 exp_x = np.exp(x)
7
8 print(f"x (radianes): {np.round(x, 4)}")
9 print(f"sin(x): {np.round(seno_x, 4)}")
10 print(f"cos(x): {np.round(coseno_x, 4)}")
11
12 # Broadcasting: escalar sobre matriz
13 matriz = np.array([[1, 2, 3], [4, 5, 6]])
14 resultado = (matriz * 2.5) + 10.0
15 print(f"\nMatriz original x 2.5 + 10:\n{resultado}")
```

9.3 Indexación Avanzada, Slicing y Máscaras Booleanas

Código Python: Filtrado Condicional con Máscaras Booleanas

```
1 datos_sensores = np.array([23.4, 45.1, 18.9, 52.6, 31.0, 64.8, 12.3])
2
3 # Creacion de mascara booleana para valores > 30.0
4 mascara_alerta = datos_sensores > 30.0
5 print(f"Mascara booleana: {mascara_alerta}")
6
7 # Filtrado directo mediante indexacion booleana
8 valores_altos = datos_sensores[mascara_alerta]
9 print(f"Valores filtrados (> 30.0): {valores_altos}")
10
11 # Reemplazo condicional con np.where()
12 estado_sensores = np.where(datos_sensores > 50.0, "CRITICO", "NORMAL")
13 print(f"Estados clasificados: {estado_sensores}")
```

9.4 Estadística Descriptiva y Álgebra Lineal

Código Python: Estadística y Multiplicación Matricial

```
1 A = np.array([[2, 1], [1, 3]], dtype=np.float64)
2 B = np.array([[4, 5], [6, 7]], dtype=np.float64)
3
4 # 1. Multiplicacion matricial (Producto punto @)
5 C = A @ B # o np.dot(A, B)
6 print(f"Producto Matricial A @ B:\n{C}")
7
8 # 2. Inversa y Determinante
9 det_A = np.linalg.det(A)
10 inv_A = np.linalg.inv(A)
11 print(f"Determinante de A: {det_A:.4f}")
12 print(f"Inversa de A:\n{inv_A}")
13
14 # 3. Estadística de un conjunto de datos
15 muestras = np.array([15.2, 16.8, 14.9, 15.5, 17.1, 15.8, 16.0])
16 print(f"\nMedia: {np.mean(muestras):.3f}")
17 print(f"Desv. Est: {np.std(muestras):.3f}")
```

```
18 print(f"Varianza: {np.var(muestras):.3f}")
19 print(f"Min / Max: {np.min(muestras)} / {np.max(muestras)}")
```

Salida en Consola

```
Shape arr_2d: (2, 3) | Dimensiones: 2 | Tipo: int32
Linspace (0 a 1, 5 pts): [0.    0.25 0.5   0.75 1.   ]
Matriz Aleatoria Uniforme:
[[13.74540119 19.50714306]
 [17.31993942 15.98658484]]

x (radianes): [0.    0.5236 0.7854 1.5708]
sin(x):       [0.    0.5   0.7071 1.    ]
cos(x):       [1.    0.866 0.7071 0.    ]

Matriz original x 2.5 + 10:
[[12.5 15.   17.5]
 [20.   22.5 25.  ]]

Mascara booleana: [False  True False  True  True  True False]
Valores filtrados (> 30.0): [45.1 52.6 31.   64.8]
Estados clasificados: ['NORMAL' 'NORMAL' 'NORMAL' 'CRITICO' 'NORMAL' 'CRITICO'
 'NORMAL']

Producto Matricial A @ B:
[[14. 17.]
 [22. 26.]]
Determinante de A: 5.0000
Inversa de A:
[[ 0.6 -0.2]
 [-0.2  0.4]]

Media:      15.900
Desv. Est:  0.725
Varianza:   0.526
Min / Max:  14.9 / 17.1
```

10 Proyecto Integrador: Sistema de Análisis Geométrico y Estadístico

Para consolidar todos los temas estudiados (f-strings, funciones, cálculos de área y volumen, listas, manipulación de strings, condicionales if, bucles con patrones for i, y NumPy), presentamos un programa completo que modela y evalúa un conjunto de piezas mecánicas tridimensionales.

Código Python: Script Maestro Integrador (analisis_solidos.py)

```
1 import math
2 import numpy as np
3
4 def calcular_propiedades_cilindro(radio: float, altura: float,
5     material: str) -> dict:
6     """Calcula area base, volumen, masa y clasificacion de un
7         cilindro."""
8     densidades = {"acero": 7.85, "aluminio": 2.70, "titanio": 4.50}
9     # g/cm3
10    mat_limpio = material.strip().lower()
11
12    if mat_limpio not in densidades:
13        raise ValueError(f"Material desconocido: '{material}'.
14            Opciones: {list(densidades.keys())}")
15
16    area_base = math.pi * (radio ** 2)
17    volumen = area_base * altura
18    masa_gramos = volumen * densidades[mat_limpio]
19
20    # Clasificacion con estructura if / elif / else
21    if masa_gramos < 500.0:
22        categoria = "Pieza Ligera"
23    elif 500.0 ≤ masa_gramos < 2000.0:
24        categoria = "Pieza Estandar"
25    else:
26        categoria = "Pieza Pesada (Uso Estructural)"
27
28    return {
29        "material": mat_limpio.upper(),
30        "radio": radio,
31        "altura": altura,
32        "area_base": area_base,
33        "volumen": volumen,
34        "masa_g": masa_gramos,
35        "categoria": categoria
36    }
37
38 # 1. Lista de especificaciones en formato string crudo
39 especificaciones_crudas = [
40     "ID:001 ; R: 3.5 ; H: 10.0 ; MAT: Acero ",
41     "ID:002 ; R: 5.0 ; H: 15.0 ; MAT: Aluminio ",
42     "ID:003 ; R: 2.0 ; H: 8.0 ; MAT: Titanio ",
43     "ID:004 ; R: 6.5 ; H: 20.0 ; MAT: Acero "
44 ]
45
46 # 2. Parseo de datos mediante manipulacion de strings y comprension
47 de listas
```

```

43 piezas_procesadas = []
44 for registro in especificaciones_crudas:
45     partes = [p.strip() for p in registro.split(";")]
46     id_pieza = partes[0].split(":")[1].strip()
47     r = float(partes[1].split(":")[1].strip())
48     h = float(partes[2].split(":")[1].strip())
49     mat = partes[3].split(":")[1].strip()
50
51     props = calcular_propiedades_cilindro(r, h, mat)
52     props["id"] = id_pieza
53     piezas_procesadas.append(props)
54
55 # 3. Analisis estadistico vectorizado con NumPy
56 volumenes_np = np.array([p["volumen"] for p in piezas_procesadas])
57 masas_np = np.array([p["masa_g"] for p in piezas_procesadas])
58
59 # 4. Reporte tabulado utilizando f-strings avanzadas
60 print("=" * 78)
61 print(f'{\'REPORTE DE CONTROL DE CALIDAD Y VOLUMETRIA\':^78}')
62 print("=" * 78)
63 print(f'{\'ID\':<6} | {\'Material\':<10} | {\'Radio\':<6} | {\'Altura\':<6} | {\'Volumen (cm3)\':>13} | {\'Masa (g)\':>10} | {\'Categoria\'}')
64 print("-" * 78)
65
66 for p in piezas_procesadas:
67     print(f"{p['id']:<6} | {p['material']:<10} | {p['radio']:>5.2f} | {p['altura']:>5.2f} | {p['volumen']:>13.2f} | {p['masa_g']:>10.2f} | {p['categoria']}"")
68
69 print("-" * 78)
70 print("Resumen Estadistico con NumPy:")
71 print(f"  Volumen Total:      {np.sum(volumenes_np):>10.2f} cm3 | Masa Total:      {np.sum(masas_np)/1000:>8.3f} kg")
72 print(f"  Volumen Promedio: {np.mean(volumenes_np):>10.2f} cm3 | Masa Promedio: {np.mean(masas_np):>8.2f} g")
73 print(f"  Volumen Maximo:   {np.max(volumenes_np):>10.2f} cm3 | Masa Maxima:   {np.max(masas_np):>8.2f} g")
74 print("=" * 78)
75
76 # 5. Generacion de un patron grafico con bucles for i para histograma visual
77 print("\nHistograma Grafico de Masa (Cada '*' representa ~500g):")
78 for i, p in enumerate(piezas_procesadas, start=1):
79     barras = int(p["masa_g"] // 500)
80     asteriscos = "*" * (barras if barras > 0 else 1)
81     print(f"  Pieza {p['id']} [{p['material']:<8}]: {asteriscos:<20} ({p['masa_g']:>7.1f} g)")

```

Salida del Proyecto Integrador

```

=====
                        REPORTE DE CONTROL DE CALIDAD Y VOLUMETRIA
=====
ID      | Material  | Radio  | Altura | Volumen (cm3) | Masa (g) | Categoria
-----

```

```

001      | ACERO      |  3.50 | 10.00 |      384.85 |    3021.03 | Pieza
      Pesada (Uso Estructural)
002      | ALUMINIO   |  5.00 | 15.00 |     1178.10 |    3180.86 | Pieza
      Pesada (Uso Estructural)
003      | TITANIO    |  2.00 |  8.00 |      100.53 |     452.39 | Pieza
      Ligera
004      | ACERO      |  6.50 | 20.00 |     2654.65 |   20838.97 | Pieza
      Pesada (Uso Estructural)
-----
Resumen Estadístico con NumPy:
Volumen Total:      4318.12 cm3 | Masa Total:      27.493 kg
Volumen Promedio:   1079.53 cm3 | Masa Promedio:   6873.31 g
Volumen Maximo:     2654.65 cm3 | Masa Maxima:     20838.97 g
=====

Histograma Grafico de Masa (Cada '*' representa ~500g):
Pieza 001 [ACERO   ]: *****          ( 3021.0 g)
Pieza 002 [ALUMINIO]: *****          ( 3180.9 g)
Pieza 003 [TITANIO ]: *                (  452.4 g)
Pieza 004 [ACERO   ]: *****          (20839.0 g)
    
```


10.1 Cuadro Resumen de Sintaxis y Referencia Rápida

Concepto	Sintaxis / Idioma Python	Resultado / Propósito
f-strings	<code>f"{var:.2f} {txt:>10}"</code>	Interpolación de variables con formato
Funciones	<code>def calc(a: float, b=10) ->float:</code>	Bloques modulares parametrizados
Área Rectángulo	<code>base * altura</code>	$A = b \cdot h$
Área Triángulo	<code>(base * altura) / 2</code>	$A = \frac{1}{2}b \cdot h$
Vol. Cilindro	<code>math.pi * (r ** 2) * h</code>	$V = \pi r^2 h$
Vol. Esfera	<code>(4/3) * math.pi * (r ** 3)</code>	$V = \frac{4}{3}\pi r^3$
Listas	<code>[x**2 for x in lista if x >0]</code>	Comprensión, mutabilidad, métodos
Strings	<code>s.strip().upper().split(";")</code>	Inmutabilidad, limpieza, partición
Condicional if	<code>if a >b: ... elif: ... else: ...</code>	Control de flujo y toma de decisiones
Bucles for i	<code>for i in range(1, n+1):</code>	Iteración acotada y patrones
NumPy Arrays	<code>np.array(), np.linspace(), arr[arr>0]</code>	Vectorización, álgebra y estadística

Tabla 4: Matriz comparativa de conceptos fundamentales de Python y NumPy.

Conclusión y Recomendaciones de Aprendizaje

El dominio conjunto de las estructuras base del lenguaje (funciones, tipos mutables e inmutables, control de flujo y cadenas) con la potencia de bibliotecas científicas como **NumPy** constituye la base fundamental para el desarrollo de software moderno en ciencia de datos, simulación numérica, inteligencia artificial e ingeniería computacional.